Esercitazione 8 - 16/11

Indice degli esercizi

Esercizio	Argomento
Esercizio 1	Inversione di un array
Esercizio 2	Ogni 2 è seguito da un 3
Esercizio 3	Riempire un array con numeri casuali
Esercizio 4	Riempire un array con numeri chiesti all'utente
Esercizio 5	Copiare un array: perché non basta `b = a`

Esercizio 1

Ripasso: Recap 1 · Gli array (indici, `length`, ciclo `for`) e Recap 2 · Array e funzioni (regola del nuovo array).

Dato un array `v`, scrivere un programma che costruisca un **nuovo array** `risultato` (senza modificare `v`) e lo stampi.

A. `risultato` contiene gli stessi elementi di `v` in ordine **inverso**. Ad esempio con `v = [1, 2, 3, 4]` si stampa `[4, 3, 2, 1]`.

B. In `risultato` gli elementi che si trovano in **posizione pari** (indici `0, 2, 4, ...`) sono presi in ordine inverso, mentre quelli in posizione dispari **restano dove sono**. Ad esempio:

- con `v = [1, 2, 3, 4, 5]` si stampa `[5, 2, 3, 4, 1]`
- con `v = [10, 11, 12, 13, 14, 15]` si stampa `[14, 11, 12, 13, 10, 15]`

Provate i programmi anche con un array di un solo elemento e con l'array vuoto.

▼ **Soluzione**

A. L'elemento in posizione `i` del risultato è quello in posizione `v.length - 1 - i` di `v`:

```
let v = [1, 2, 3, 4];

let risultato = [];
for (let i = 0; i < v.length; i++) {
  risultato[i] = v[v.length - 1 - i];
}

console.log(risultato);                          // [ 4, 3, 2, 1 ]
console.log(v);                                  // [ 1, 2, 3, 4 ]: v non è cambiato
```

Con `i = 0` si prende `v[length - 1]` (l'ultimo), con `i = length - 1` si prende `v[0]`.

B. Le posizioni dispari si copiano uguali. Per le posizioni pari serve la stessa idea di A, ma lo «specchio» va fatto **tra le sole posizioni pari**: bisogna conoscere l'**ultima posizione pari**, che è `length - 1` se la lunghezza è dispari e `length - 2` se è pari.

```
let v = [10, 11, 12, 13, 14, 15];

let ultimoPari;
if (v.length % 2 == 1) {
  ultimoPari = v.length - 1;
} else {
  ultimoPari = v.length - 2;
}

let risultato = [];
for (let i = 0; i < v.length; i++) {
  if (i % 2 == 0) {
    risultato[i] = v[ultimoPari - i];            // specchio tra le posizioni pari
  } else {
    risultato[i] = v[i];                         // le posizioni dispari non si muovono
  }
}

console.log(risultato);                          // [ 14, 11, 12, 13, 10, 15 ]
```

Le posizioni pari sono `0, 2, 4` (con valori `10, 12, 14`): `ultimoPari` vale `4`, quindi in posizione `0` va `v[4]`, in posizione `2` va `v[2]` (resta lì) e in posizione `4` va `v[0]`. Con `v = [1, 2, 3, 4, 5]` si ha `ultimoPari = 4` e si stampa `[ 5, 2, 3, 4, 1 ]`; con `[7]` si stampa `[ 7 ]`; con `[]` il ciclo non parte e si stampa `[]`.

Esercizio 2

Ripasso: Recap 1 · Gli array; per la variabile flag, l'esercizio di ricerca dello zero dell'esercitazione 7.

Dato un array `v`, scrivere un programma che stampi `true` se **ogni** `2` presente nell'array è **immediatamente seguito** da un `3`, `false` altrimenti. Se non ci sono `2` la risposta è `true`; un `2` in ultima posizione non è seguito da niente, quindi dà `false`.

Provate il programma con questi array:

v	cosa stampa
`[1, 2, 3, 5, 2, 3]`	`true`
`[2, 4, 3]`	`false`
`[5, 2, 3, 2]`	`false`
`[1, 5, 7]`	`true`
`[2, 2, 3]`	`false`
`[]`	`true`

▼ **Soluzione**

È la solita **variabile flag**: si parte da `true` e si cambia idea solo se si trova un controesempio, cioè un `2` che **non** è seguito da un `3`. Per guardare l'elemento successivo si usa `v[i + 1]`: se `i` è l'ultima posizione, `v[i + 1]` non esiste e vale `undefined`, che è diverso da `3`, quindi il caso del `2` in fondo è già gestito.

```
let v = [1, 2, 3, 5, 2, 3];

let ok = true;
for (let i = 0; i < v.length; i++) {
  if (v[i] == 2 && v[i + 1] != 3) {            // un 2 non seguito da un 3
    ok = false;
  }
}

console.log(ok);                               // true
```

Per gli altri array della tabella si ottiene: `[2, 4, 3]` → `false`; `[5, 2, 3, 2]` → `false` (l'ultimo `2` non ha un successivo); `[1, 5, 7]` → `true` (nessun `2`); `[2, 2, 3]` → `false` (il primo `2` è seguito da un `2`); `[]` → `true`.

Due errori frequenti:

- mettere `ok = true` in un `else` dentro il ciclo: il flag verrebbe rimesso a `true` dagli elementi successivi, e conterebbe solo l'**ultimo** controllo;

- scrivere `v[i + 1] == 3` come condizione di errore, cioè invertire il test: il controesempio è un `2` seguito da qualcosa di **diverso** da `3`.

  In alternativa si può restituire `false` appena si trova il controesempio e `true` **dopo** il ciclo.

Esercizio 3

Ripasso: Recap 2 · Array e funzioni; la funzione `contiene` è nell'Esercizio 11 dell'esercitazione 7.

Funzioni che **costruiscono** un array di numeri casuali rispettando una regola. Ricordate che un intero casuale tra `min` e `max` (estremi inclusi) è `Math.floor(Math.random() * (max - min + 1)) + min`.

A. Scrivere la funzione `casualiPari(n, min, max)` che restituisce un array di `n` numeri casuali **pari** compresi tra `min` e `max`.

B. Scrivere la funzione `casualiDistinti(n, min, max)` che restituisce un array di `n` numeri casuali compresi tra `min` e `max` **tutti diversi tra loro**. Suggerimento: estraete un numero con un `do ... while` finché non ne trovate uno che non è già nell'array.

Provatele, ad esempio, con `casualiPari(8, 1, 20)` e `casualiDistinti(5, 1, 10)`.

▼ **Soluzione**

A. Il modo più semplice per ottenere un pari è **estrarre finché non esce un pari**, con un `do ... while`:

```
function casualiPari(n, min, max) {
  let risultato = [];
  for (let i = 0; i < n; i++) {
    let x;
    do {
      x = Math.floor(Math.random() * (max - min + 1)) + min;
    } while (x % 2 != 0);                      // si ripete finché x è dispari
    risultato[i] = x;
  }
  return risultato;
}

console.log(casualiPari(8, 1, 20));            // ad esempio [ 12, 4, 18, 4, 2, 20, 10, 6 ]
```

Si può usare anche una formula che produce **direttamente** un pari (si estrae un numero casuale e lo si moltiplica per `2`), ma bisogna stare attenti agli estremi: con `min` e `max` dispari il primo e l'ultimo pari possibili sono diversi da `min` e `max`. Il `do ... while` evita questo problema. Attenzione

però: se tra `min` e `max` **non c'è nessun pari** (ad esempio `min = max = 7`) il ciclo non finisce mai.

B. La regola «diverso da tutti gli altri» si controlla con la funzione `contiene` dell'esercitazione 7, applicata al risultato che si sta costruendo:

```
function contiene(v, x) {
  for (let i = 0; i < v.length; i++) {
    if (v[i] == x) {
      return true;
    }
  }
  return false;
}

function casualiDistinti(n, min, max) {
  let risultato = [];
  for (let i = 0; i < n; i++) {
    let x;
    do {
      x = Math.floor(Math.random() * (max - min + 1)) + min;
    } while (contiene(risultato, x));           // si ripete finché x c'è già
    risultato[risultato.length] = x;
  }
  return risultato;
}

console.log(casualiDistinti(5, 1, 10));         // ad esempio [ 7, 2, 9, 4, 10 ]
```

Qui si assegna a `risultato[risultato.length]` (e non a `risultato[i]`): è lo stesso, ma rende evidente che si aggiunge **in fondo**, e che `contiene` guarda solo gli elementi già inseriti.

Attenzione: se i valori possibili sono **meno** di `n` (ad esempio `casualiDistinti(5, 1, 3)`) non ci sono abbastanza numeri diversi e il programma non termina. Una funzione robusta controllerebbe prima che `max - min + 1 >= n`.

Esercizio 4

Ripasso: Recap 2 · Array e funzioni.

Funzioni che **costruiscono** un array con numeri **chiesti all'utente**. Si legge con `prompt` e si converte con `Number`, come negli esercizi precedenti.

A. Scrivere la funzione `leggiArray(n)` che chiede all'utente `n` numeri e restituisce l'array che li contiene, nell'ordine in cui sono stati inseriti.

B. Scrivere la funzione `leggiFinoAZero()` che chiede numeri **finché l'utente non inserisce** `0` e restituisce l'array dei numeri inseriti, **senza** lo zero finale. Non si sa in anticipo quanti saranno.

C. Scrivere la funzione `leggiPositivi(n)` che chiede all'utente `n` numeri **positivi**: se l'utente ne inserisce uno non positivo, il programma lo richiede finché non è valido.

Per ciascuna scrivete poi un breve programma che la usa e stampa l'array letto.

▼ **Soluzione**

A. Si sa quante volte ripetere: un ciclo `for` che a ogni giro aggiunge il numero letto:

```
function leggiArray(n) {
  let risultato = [];
  for (let i = 0; i < n; i++) {
    risultato[i] = Number(prompt("Numero " + (i + 1) + ": "));
  }
  return risultato;
}

let v = leggiArray(4);
console.log(v);                                  // ad esempio [ 5, -2, 9, 5 ]
```

B. Non si sa quante volte ripetere: serve un `while` con **sentinella** (lo zero), come nell'esercitazione 6. Si legge **prima** del ciclo e **di nuovo** alla fine di ogni giro:

```
function leggiFinoAZero() {
  let risultato = [];
  let x = Number(prompt("Numero (0 per finire): "));
  while (x != 0) {
    risultato[risultato.length] = x;             // lo zero finale non si aggiunge
    x = Number(prompt("Numero (0 per finire): "));
  }
  return risultato;
}

let w = leggiFinoAZero();
console.log(w);                                  // con 4, 7, 0 si stampa [ 4, 7 ]
```

Qui si assegna a `risultato[risultato.length]`, e **non** a `risultato[i]`: non c'è un contatore `i`, e si aggiunge sempre in fondo. Se l'utente scrive subito `0` si restituisce l'array vuoto `[]`.

C. È la validazione dell'input con `do ... while` (esercitazione 4), ripetuta per ognuno degli `n` numeri:

```
function leggiPositivi(n) {
  let risultato = [];
  for (let i = 0; i < n; i++) {
    let x;
    do {
      x = Number(prompt("Numero positivo " + (i + 1) + ": "));
    } while (x <= 0);                          // si richiede finché non è positivo
    risultato[i] = x;
  }
  return risultato;
}

let p = leggiPositivi(3);
console.log(p);
```

La variabile `x` va dichiarata **fuori** dal `do ... while` ma **dentro** il `for`: così il `while (x <= 0)` la vede, e a ogni giro del `for` se ne usa una nuova.

Esercizio 5

Ripasso: Recap 2 · Array e funzioni (la parte sul passaggio per riferimento).

Un array **non si copia** con l'assegnazione. Vediamo perché e come si fa.

A. Dite cosa stampa questo programma, **senza eseguirlo**. Poi spiegate con le vostre parole perché `a` è cambiato anche se abbiamo modificato solo `b`.

```
let a = [1, 2, 3];
let b = a;
b[0] = 99;
console.log(a);
console.log(b);
console.log(a == b);
```

B. Volevamo **conservare** `a` e lavorare su una copia. Scrivere la funzione `copia(v)` che restituisce un **nuovo array** con gli stessi elementi di `v`, e usarla per correggere il programma di A: dopo la modifica di `b`, `a` deve essere ancora `[1, 2, 3]`.

▼ Soluzione

A. Stampa `[ 99, 2, 3 ]`, poi `[ 99, 2, 3 ]`, poi `true`.

Una variabile che contiene un array **non contiene gli elementi**: contiene un **riferimento**, cioè l'«indirizzo» dell'array in memoria. Con `let b = a;` si copia solo il riferimento: `a` e `b` sono **due**

nomi per lo stesso array. Modificare l'array tramite `b` lo cambia anche per `a`, perché c'è un solo array. Per questo `a == b` è `true`: sono lo stesso array.

È diverso dai numeri: con `let y = x;` si copia il **valore**, e cambiare `y` non tocca `x`.

B. Per avere una copia vera bisogna **creare un array nuovo** e riempirlo con gli elementi di `v`:

```
function copia(v) {
  let risultato = [];
  for (let i = 0; i < v.length; i++) {
    risultato[i] = v[i];
  }
  return risultato;
}

let a = [1, 2, 3];
let b = copia(a);                              // array nuovo, non un secondo nome per a
b[0] = 99;
console.log(a);                                // [ 1, 2, 3 ]
console.log(b);                                // [ 99, 2, 3 ]
console.log(a == b);                           // false: sono due array diversi
```

È anche il motivo per cui le nostre funzioni **costruiscono un array nuovo** invece di modificare quello ricevuto: un parametro array è un riferimento allo stesso array di chi chiama.

Ludovica Pannitto - Università di Pisa - CC BY-NC-SA 4.0